

NLP-RS Developer Documentation

Sep 24, 2026 · @owoabasi

Technical build specification for the NLP-Driven Curriculum Recommendation System (NLP-RS) — the software artefact for the final-year project *Development of an NLP-Driven Curriculum Recommendation System for NUC CCMAS 30% Localized Curriculum Customisation*, University of Uyo, Department of Computer Science. This document gives a developer everything needed to start building immediately: architecture, schema, pipeline spec, API contract, frontend structure, setup steps, testing plan, and a sprint-by-sprint build order.

1. System Overview and Objectives

What it is. NLP-RS is a web application that helps a university curriculum planning committee decide what to put in the NUC CCMAS "30% locally customised" portion of a programme. It ingests job adverts, regional policy documents, and academic literature, runs them through an NLP pipeline, and produces ranked, evidence-backed course-topic recommendations, each checked against the existing NUC 70% core curriculum to avoid duplication.

Primary users.

1. Curriculum Planner (academic staff on the curriculum committee) — uploads documents, configures and runs analysis sessions, reviews and accepts/rejects recommendations, maps accepted recommendations to course codes.
2. Administrator — manages users, the NUC 70% core reference repository, and system-wide NLP parameter defaults.

Core objectives the build must satisfy (these map directly to the project's Chapter One/Three objectives and become the acceptance criteria for testing in Section 10):

- Ingest and manage a multi-source document library (job ads, policy PDFs, institutional handbooks, literature).
- Run an 8-stage NLP pipeline (Section 5) producing keyword, entity, semantic, and topic-level features.
- Detect semantic overlap between a candidate 30% topic and the existing 70% core curriculum (target: SBERT cosine similarity threshold 0.80).
- Score and rank candidate curriculum topics using a weighted composite formula (Section 6).

- Present recommendations with a transparent, traceable evidence trail (every score must be explainable back to source text).
- Support a structured curriculum-mapping workflow (course code, credit units, learning outcomes) for accepted recommendations.
- Generate a NUC-accreditation-ready curriculum planning report (PDF/DOCX export).
- Be usable by non-technical academic staff (System Usability Scale target: ≥ 70).

Out of scope for v1: real-time labour-market API integration, multi-institution/multi-tenant support, and non-Computing disciplines — these are documented as future work in the project report and should not block the v1 build.

2. Tech Stack and Architecture

Three-tier architecture, chosen for a clean separation between the NLP-heavy Python backend and a conventional CRUD web frontend.

Tier	Technology	Why
Presentation	Vue.js 3 (Composition API) + Bootstrap 5	Component-based SPA, fast to build forms/dashboards; Bootstrap gives accessible, responsive UI with minimal custom CSS
Logic	Python 3.11 + Flask 3.x	Python has the mature NLP ecosystem (spaCy, sentence-transformers, BERTopic); Flask is lightweight enough for a project-scale REST API
Data	PostgreSQL 15 + SQLAlchemy ORM	Relational integrity for sessions/recommendations/mappings; JSONB columns for flexible NLP output storage (embeddings, entity lists)
NLP libraries	spaCy (NER), scikit-learn (TF-IDF), sentence-transformers (SBERT), BERTopic (+ UMAP, HDBSCAN)	Matches the techniques justified in the project's Chapter Two/Three literature review
Async task queue	Celery + Redis	The 8-stage pipeline can take up to ~60s per session; runs as a background job so the UI stays responsive
Auth	Flask-JWT-Extended	Stateless auth for the SPA-to-API pattern; roles: Planner, Administrator

High-level flow.

flowchart LR

```

A[Curriculum Planner] -->|uploads docs, configures session| B[Vue.js SPA]
B -->|REST/JSON + JWT| C[Flask API]
C -->|enqueue| D[Celery Worker: 8-stage NLP Pipeline]
D -->|reads/writes| E[(PostgreSQL)]
C -->|reads/writes| E
D -->|cache/queue| F[(Redis)]
C -->|generates| G[PDF/DOCX Report]
B -->|renders| H[Recommendation Dashboard]
```

Deployment shape. Frontend built as static assets served via Nginx; Flask API behind Gunicorn; Celery worker as a separate process; PostgreSQL and Redis as managed services or Docker containers. See Section 11 for environment-specific detail.

3. Repository Structure

```

nlp-rs/
├── backend/
│   ├── app/
│   │   ├── __init__.py          # Flask app factory
│   │   ├── config.py           # env-driven config classes
│   │   ├── extensions.py       # db, jwt, celery init
│   │   └── models/             # SQLAlchemy models (one file per entity,
Section 4)
│   │       ├── document.py
│   │       ├── analysis_session.py
│   │       ├── nlp_result.py
│   │       ├── recommendation.py
│   │       ├── curriculum_map.py
│   │       ├── user.py
│   │       └── audit_log.py
│   └── api/                     # Flask blueprints, one per resource
(Section 7)
│       ├── documents.py
│       ├── sessions.py
│       ├── recommendations.py
│       ├── curriculum_maps.py
│       ├── reports.py
│       └── auth.py
│   └── nlp/                     # the 8-stage pipeline (Section 5)
│       ├── pipeline.py         # orchestrator
│       ├── ingestion.py        # stage 1
│       ├── preprocessing.py    # stage 2
│       ├── tfidf.py            # stage 3
│       ├── ner.py              # stage 4
│       ├── sbert.py            # stage 5
│       ├── bertopic_stage.py   # stage 6
│       ├── scoring.py          # stage 7 (Section 6 formula)
│       └── output.py           # stage 8
│   └── tasks.py                 # Celery task definitions wrapping
pipeline.py
│   ├── reports/                # PDF/DOCX generation helpers
│   ├── migrations/             # Alembic migrations
│   ├── tests/
│   │   ├── unit/
│   │   ├── integration/
│   │   └── fixtures/          # sample_job_ads.pdf,
sample_core_curriculum.csv, etc.
│   ├── requirements.txt
│   └── celery_worker.py

```

```

├── wsgi.py
├── frontend/
│   ├── src/
│   │   ├── main.js
│   │   ├── router/index.js
│   │   ├── store/                # Pinia stores (Section 8)
│   │   ├── views/               # page-level components
│   │   │   ├── LoginView.vue
│   │   │   ├── DocumentLibraryView.vue
│   │   │   ├── SessionConfigView.vue
│   │   │   ├── RecommendationDashboardView.vue
│   │   │   ├── CurriculumMapView.vue
│   │   │   ├── ReportsView.vue
│   │   │   └── AdminView.vue
│   │   ├── components/          # reusable widgets (RecommendationCard,
ScoreBreakdown, etc.)
│   │   └── api/                 # axios client wrappers, one per backend
resource
│   ├── package.json
│   └── vite.config.js
├── docker-compose.yml           # postgres, redis, backend, worker,
frontend
├── .env.example
└── README.md

```

Backend and frontend are separate deployable units communicating only over the REST API in Section 7 — no server-rendered templates.

4. Database Schema

Seven core entities. All primary keys are UUIDs; timestamps (`created_at` , `updated_at`) are omitted from the table below for brevity but should be present on every entity.

User

Field	Type	Notes
id	UUID PK	
name	VARCHAR(120)	
email	VARCHAR(255) UNIQUE	
password_hash	VARCHAR(255)	bcrypt
role	ENUM('planner','admin')	

Document

Field	Type	Notes
id	UUID PK	
filename	VARCHAR(255)	
file_path	VARCHAR(500)	storage location (local/S3)
doc_type	ENUM('job_ad','policy','handbook','literature','core_curriculum')	
uploaded_by	UUID FK → User.id	
status	ENUM('pending','processed','failed')	

AnalysisSession

Field	Type	Notes
id	UUID PK	
name	VARCHAR(120)	
created_by	UUID FK → User.id	
topic_count	INTEGER	BERTopic parameter, default 10
overlap_threshold	FLOAT	default 0.80 (SBERT cosine similarity)
status	ENUM('draft','running','completed','failed')	

DocumentSession (junction table, many-to-many Document ↔ AnalysisSession)

Field	Type	Notes
id	UUID PK	
document_id	UUID FK → Document.id	
session_id	UUID FK → AnalysisSession.id	
order_index	INTEGER	

NLPResult

Field	Type	Notes
id	UUID PK	
document_session_id	UUID FK → DocumentSession.id	
tfidf_scores	JSONB	top-weighted terms
entities	JSONB	NER output, list of {text, label}
embedding	JSONB / VECTOR(768)	SBERT sentence embedding (use pgvector extension if available, else JSONB)
topic_id	INTEGER	BERTopic cluster assignment

Recommendation

Field	Type	Notes
id	UUID PK	
session_id	UUID FK → AnalysisSession.id	
title	VARCHAR(255)	
description	TEXT	
ner_score	FLOAT	component (weight 0.40)
bertopic_score	FLOAT	component (weight 0.35)
novelty_score	FLOAT	component (weight 0.25)
composite_score	FLOAT	Section 6 formula
overlap_status	ENUM('clear','flagged')	vs NUC 70% core
overlap_similarity	FLOAT	SBERT cosine similarity to nearest core course
decision	ENUM('pending','accepted','rejected')	
planner_notes	TEXT	nullable

CurriculumMap

Field	Type	Notes
id	UUID PK	
recommendation_id	UUID FK → Recommendation.id	
course_code	VARCHAR(20)	e.g. UUY-CSC 411
course_title	VARCHAR(255)	
credit_units	INTEGER	
prerequisites	JSONB	array of course codes
learning_outcomes	JSONB	array of strings

AuditLog

Field	Type	Notes
id	UUID PK	
user_id	UUID FK → User.id	
action_type	VARCHAR(50)	e.g. 'upload_document', 'run_session', 'accept_recommendation'
entity_type	VARCHAR(50)	
entity_id	UUID	
detail	TEXT	

Relationships: User 1–N Document; User 1–N AnalysisSession; AnalysisSession N–N Document (via DocumentSession); DocumentSession 1–1 NLPResult; AnalysisSession 1–N Recommendation; Recommendation 1–1 CurriculumMap (nullable until accepted); User 1–N AuditLog.

5. NLP Pipeline Specification

Orchestrated as a Celery task (`nlp/pipeline.py::run_pipeline(session_id)`) that runs all 8 stages sequentially per document, then aggregates at Stage 7. Each stage writes its output onto the `NLPResult` row for that document-session pair.

Stage	Name	Library	Input	Output
1	Document Ingestion	PyPDF2 / python-docx / pandas	raw file (PDF/DOCX/CSV)	plain text string
2	Preprocessing	spaCy (en_core_web_sm)	raw text	tokenised, lower-cased, stop-word-filtered, lemmatised text
3	TF-IDF Extraction	scikit-learn TfidfVectorizer	preprocessed corpus	top-N weighted terms per document
4	Named Entity Recognition	spaCy custom pipeline + rule-based patterns for TECHNOLOGY/SKILL/METHODOLOGY entities	preprocessed text	list of {entity_text, label, frequency}
5	Sentence Embedding	sentence-transformers (all-mpnet-base-v2 or similar SBERT model)	document/passage text	768-dim dense vector
6	Topic Modelling	BERTopic (UMAP + HDBSCAN + c-TF-IDF)	corpus embeddings	topic_id per document + topic keyword sets
7	Weighted Recommendation Scoring	custom (scoring.py)	NER freq, BERTopic relevance, SBERT novelty	composite_score per candidate topic (Section 6)
8	Output Generation	custom	scored, ranked candidates	Recommendation rows persisted, ready for dashboard

Semantic overlap detection (runs alongside Stage 7): for every candidate recommendation, compute SBERT cosine similarity against every course description in the NUC 70% core reference corpus (Document rows with doc_type='core_curriculum'). If max similarity $\geq$ overlap_threshold (default 0.80), mark overlap_status='flagged' ; else 'clear' .

Pseudocode for the orchestrator:

```
def run_pipeline(session_id):
    session = AnalysisSession.query.get(session_id)
    docs = get_session_documents(session_id)

    for doc in docs:
        text = ingest(doc.file_path)                # Stage 1
        clean = preprocess(text)                     # Stage 2
        tfidf_scores = extract_tfidf(clean, corpus=docs) # Stage 3
        entities = extract_entities(clean)            # Stage 4
        embedding = embed_sentence(clean)              # Stage 5
        save_nlp_result(doc, tfidf_scores, entities, embedding)

    topics = run_bertopic(all_embeddings(docs), n_topics=session.topic_count)
    # Stage 6
    assign_topics(docs, topics)
```

```

candidates = build_candidate_topics(docs, topics)
for c in candidates:
    c.ner_score = normalize(entity_frequency(c))
    c.bertopic_score = normalize(topic_relevance(c))
    c.novelty_score = compute_novelty(c, core_corpus_embeddings)
    c.composite_score = score(c) # Stage 7, Section 6
    c.overlap_similarity, c.overlap_status = detect_overlap(c,
core_corpus_embeddings, session.overlap_threshold)

persist_recommendations(session, candidates) # Stage 8
session.status = 'completed'

```

Target performance benchmarks (from the project's evaluation plan — use as pipeline acceptance tests): NER precision > 0.80, recall > 0.75, F1 > 0.75; BERTopic topic coherence (Cv) > 0.50; SBERT overlap detection AUC-ROC > 0.80; full pipeline processing time < 60 seconds for a typical session; recommendation coverage ≥ 85% (share of accepted recommendations judged relevant by planners in UAT).

6. Recommendation Scoring Algorithm

Composite score is a weighted sum of three normalised (0-1) components:

$$S_c = 0.40 \cdot N_c + 0.35 \cdot B_c + 0.25 \cdot V_c$$

Where for candidate topic c :

- **N_c (NER demand frequency, weight 0.40):** normalised frequency of the candidate's associated entities (skills/technologies/methodologies) across the job-advertisement corpus. $N_c = \text{count}(\text{entities}_c) / \text{max}(\text{count}(\text{entities}) \text{ across all candidates})$.
- **B_c (BERTopic thematic relevance, weight 0.35):** how strongly the candidate's topic cluster is represented and coherent in the corpus, derived from BERTopic's c-TF-IDF score for the topic, normalised across candidates.
- **V_c (semantic novelty, weight 0.25):** $V_c = 1 - \text{max_similarity}(c, \text{core_corpus})$ — higher when the candidate is more semantically distinct from the existing NUC 70% core (i.e. it would add new content rather than duplicate).

Candidates are ranked descending by S_c and the top 20 are surfaced to the Curriculum Planner for review (Use Case 3 in Section 7/8).

Worked example (also used in the project's Appendix D as a traceability illustration):

Input job-ad passage: "We are seeking a Junior Software Developer proficient in Python, REST API design, and relational database management (PostgreSQL preferred). Familiarity with version control (Git), Agile development practices, and basic cloud deployment (AWS or Azure) is required."

Stage	Output
TF-IDF	python (0.41), rest api (0.35), postgresql (0.33), agile (0.29), cloud deployment (0.27), version control (0.24)
NER	TECHNOLOGY: Python, PostgreSQL, Git, AWS, Azure; METHODOLOGY: Agile; SKILL: REST API design, cloud deployment
SBERT embedding	768-dim vector
Overlap check	cosine similarity 0.34 vs nearest core course ("Introduction to Programming II") — below 0.80 threshold → <code>clear</code>
Composite score	$0.40(0.85) + 0.35(0.72) + 0.25(0.66) = \mathbf{0.78}$ for candidate "Cloud-Native Application Development with Python"

Implementation note: weights (0.40/0.35/0.25) should be stored as configurable constants (or per-session overrides on `AnalysisSession`, if time allows) rather than hard-coded, so the evaluation phase can test sensitivity to different weightings.

7. REST API Specification

Base URL: `/api/v1`. All endpoints except `/auth/login` require `Authorization: Bearer <JWT>`. Responses are JSON; errors follow `{"error": {"code": str, "message": str}}`.

Auth

Method	Endpoint	Body	Response
POST	<code>/auth/login</code>	<code>{email, password}</code>	<code>{access_token, user: {id, name, role}}</code>
POST	<code>/auth/refresh</code>	— (refresh token)	<code>{access_token}</code>

Documents

Method	Endpoint	Body / Params	Response
GET	/documents	?doc_type=&status=	[Document]
POST	/documents	multipart file upload + doc_type	Document (status pending)
GET	/documents/{id}	—	Document
DELETE	/documents/{id}	—	204

Analysis Sessions

Method	Endpoint	Body / Params	Response
GET	/sessions	—	[AnalysisSession]
POST	/sessions	{name, document_ids[], topic_count, overlap_threshold}	AnalysisSession (status draft)
POST	/sessions/{id}/run	—	202 {task_id} — enqueues Celery pipeline task
GET	/sessions/{id}	—	AnalysisSession incl. status (poll for completed)
GET	/sessions/{id}/status	—	{status, progress_pct} — lightweight polling endpoint

Recommendations

Method	Endpoint	Body / Params	Response
GET	/sessions/{id}/recommendations	? sort=score&limit=20	[Recommendation] incl. score breakdown and evidence
GET	/recommendations/{id}	—	Recommendation (full detail incl. source excerpts)
PATCH	/recommendations/{id}	{decision: 'accepted'}	'rejected', planner_notes'

Curriculum Mapping

Method	Endpoint	Body / Params	Response
POST	/recommendations/{id}/map	{course_code, course_title, credit_units, prerequisites[], learning_outcomes[]}	CurriculumMap
GET	/curriculum-maps	?session_id=	[CurriculumMap]

Reports

Method	Endpoint	Body / Params	Response
POST	/sessions/{id}/report	{format: 'pdf'	'docx'}
GET	/reports/{task_id}	—	file download once ready, else 202 {status: 'processing'}

Admin

Method	Endpoint	Body / Params	Response
GET / POST	/admin/users	{name, email, role}	[User] / User
PATCH	/admin/users/{id}	{role}	User
GET / POST	/admin/core-curriculum	multipart upload	[Document] filtered doc_type='core_curriculum'
GET	/admin/audit-logs	? user_id=&from=&to=	[AuditLog]

All mutating endpoints (POST/PATCH/DELETE) write an `AuditLog` row inside the same transaction.

8. Frontend Structure

Vue.js 3 SPA (Composition API + `<script setup>`), Vite build, Pinia for state, Vue Router for navigation, Bootstrap 5 for layout/components, Axios for API calls.

Routes

Path	View	Access	Purpose
<code>/login</code>	LoginView	public	JWT auth
<code>/documents</code>	DocumentLibraryView	planner, admin	upload/list/delete documents
<code>/sessions</code>	SessionConfigView (list + create)	planner, admin	create/configure/run analysis sessions
<code>/sessions/:id</code>	RecommendationDashboardView	planner, admin	ranked recommendations, accept/reject, score breakdown
<code>/sessions/:id/map/:recId</code>	CurriculumMapView	planner, admin	map an accepted recommendation to a course code
<code>/sessions/:id/report</code>	ReportsView	planner, admin	generate/download report
<code>/admin</code>	AdminView	admin only	users, core curriculum repository, audit log

Pinia stores

- `auth.js` — JWT token, current user, login/logout actions.
- `documents.js` — document list, upload progress, CRUD actions.
- `sessions.js` — session list/detail, run/poll status (polls `/sessions/{id}/status` every 3s while `running`).
- `recommendations.js` — recommendation list for the active session, accept/reject actions, optimistic UI update.

Key components

- `RecommendationCard.vue` — title, composite score, accept/reject buttons, expandable evidence panel.
- `ScoreBreakdown.vue` — visual bar breakdown of the three weighted components (NER/BERTopic/novelty) for one recommendation — use the dataviz skill's stacked/grouped-bar guidance if built as a chart rather than plain progress bars.

- `OverlapBadge.vue` — green "Clear" / amber "Flagged" badge with the similarity score on hover.
- `DocumentUploader.vue` — drag-and-drop multi-file upload with type selector and progress bar.
- `CurriculumMapForm.vue` — course code, credit units, prerequisites (tag input), learning outcomes (repeatable text fields).
- `SessionProgressBar.vue` — polls session status, shows pipeline stage progress during running.

UX note tied to the SUS target (≥ 70): minimise steps between "session completed" and "see recommendations" — redirect straight to RecommendationDashboardView on completion; keep the accept/reject action a single click with undo, not a confirmation dialog.

9. Environment Configuration and Local Setup

Prerequisites: Python 3.11, Node.js 18+, PostgreSQL 15, Redis 7, Docker (optional but recommended).

`.env` variables

```
FLASK_ENV=development
SECRET_KEY=change-me
JWT_SECRET_KEY=change-me-too
DATABASE_URL=postgresql://nlprs_user:password@localhost:5432/nlprs_db
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/0
UPLOAD_FOLDER=./uploads
MAX_CONTENT_LENGTH=20971520
SBERT_MODEL_NAME=all-mpnet-base-v2
SPACY_MODEL=en_core_web_sm
DEFAULT_OVERLAP_THRESHOLD=0.80
DEFAULT_TOPIC_COUNT=10
VITE_API_BASE_URL=http://localhost:5000/api/v1
```

Backend setup

```
cd backend
python3.11 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python -m spacy download en_core_web_sm
flask db upgrade           # run Alembic migrations
flask run                  # dev server on :5000
```

In a second terminal, start the Celery worker (required for pipeline runs):

```
cd backend && source venv/bin/activate
celery -A celery_worker.celery worker --loglevel=info
```

Frontend setup

```
cd frontend
npm install
npm run dev                # dev server on :5173, proxies /api to
backend
```

Docker Compose (all services at once)

```
docker compose up --build
```

`docker-compose.yml` should define services: `postgres`, `redis`, `backend` (Gunicorn), `worker` (Celery), `frontend` (Nginx serving the Vite build).

Seed data: load `tests/fixtures/sample_core_curriculum.csv` (the real UUY-CSC/NUC course list from the project's Chapter Four) as the initial `doc_type='core_curriculum'` reference set, and `tests/fixtures/sample_job_ads.pdf` as a starter corpus for local testing of the pipeline before real job-advert scraping is wired up.

requirements.txt (core packages)

```
Flask==3.0.*
Flask-SQLAlchemy==3.1.*
Flask-JWT-Extended==4.6.*
Flask-Cors==4.0.*
alembic==1.13.*
celery==5.3.*
redis==5.0.*
psycpg2-binary==2.9.*
spacy==3.7.*
scikit-learn==1.4.*
sentence-transformers==2.6.*
bertopic==0.16.*
umap-learn==0.5.*
hdbscan==0.8.*
PyPDF2==3.0.*
python-docx==1.1.*
pandas==2.2.*
reportlab==4.1.*
```


10. Testing Strategy

Four layers, matching the project's Chapter Four testing section and feeding its Results and Discussion.

Unit testing (`pytest` , backend `tests/unit/`)

- Each NLP stage function tested in isolation with fixed sample input → assert expected shape of output (not exact ML output, since models are non-deterministic across versions, but assert score ranges, entity label sets, and vector dimensionality).
- Scoring formula (`scoring.py`) tested with hand-computed expected composite scores for known component inputs (use the worked example in Section 6 as a golden test case).
- SQLAlchemy model constraints (foreign keys, enums, cascade deletes).

Integration testing (`pytest` + test DB, `tests/integration/`)

- Full pipeline run against `sample_job_ads.pdf` + `sample_core_curriculum.csv` fixtures → assert a `Recommendation` set is produced, scores are within $[0,1]$, and at least one known-overlapping candidate is correctly flagged.
- API endpoint tests (Flask test client) for every route in Section 7: auth required, correct status codes, correct JSON shape, audit log rows created on mutations.
- Celery task tests using `task_always_eager=True` in the test config.

System testing

- End-to-end flow through the deployed frontend + backend: login → upload documents → create session → run → review recommendations → accept → map to course → generate report. Can be scripted with Playwright/Cypress if time allows, otherwise manual per the UAT script below.

User Acceptance Testing (UAT)

- Recruit a purposive sample of Department of Computer Science faculty (the project's Appendix B task script is the exact script to run: create session, review top 5 recommendations, accept 3/reject 2, complete a curriculum mapping form, generate a report).
- Administer the System Usability Scale (10-item, Appendix C) immediately after the task session; target mean SUS score ≥ 70 ("good" per Bangor, Kortum and Miller, 2008).
- Record qualitative feedback (the two open-ended questions added in Appendix C) to feed Chapter Four's discussion section.

NLP quality benchmarks to validate against real or representative data before UAT: NER precision/recall/F1, BERTopic coherence (Cv), SBERT overlap AUC-ROC — targets as listed in Section 5. Run these as a scripted evaluation notebook

(`tests/evaluation/nlp_benchmarks.ipynb`) against a labelled subset of the corpus, not as part of the automated test suite.

11. Deployment Plan

For a final-year project, a low-cost single-server or free-tier deployment is appropriate — avoid over-engineering this section beyond what the demo/defence needs.

Recommended path

Component	Option	Notes
Backend + Worker + Postgres + Redis	Single VPS (e.g. a low-cost droplet/VM) via Docker Compose	Simplest to demo and defend; matches the <code>docker-compose.yml</code> in Section 3/9
Frontend	Same VPS behind Nginx, or a static host (Netlify/Vercel) pointed at the VPS API	Static hosting is free and simplifies HTTPS via the host's own certificate
Alternative (no server cost)	Render.com / Railway.app free/hobby tiers for backend + managed Postgres/Redis add-ons	Good fallback if a VPS isn't available; slower cold starts on free tiers

Environments: `development` (local, Section 9), `staging / demo` (for supervisor review and UAT sessions), `production` is likely unnecessary for a final-year project unless the Department wants to keep running it after submission — confirm with the supervisor whether staging alone is sufficient.

CI (minimal, optional but recommended)

- GitHub Actions workflow: on push → run backend `pytest` suite + frontend `npm run build` (catches breakage before demo day). A deploy step (SSH + `docker compose pull && up -d`, or a Render/Railway auto-deploy hook) can be added once staging is live.

Operational notes

- Back up the PostgreSQL database before every UAT session (the evaluation data in Chapter Four depends on it).
- Set `MAX_CONTENT_LENGTH` and Celery task time limits conservatively so a bad upload or a stuck pipeline run cannot hang the demo.

- Keep an `.env.demo` with the SBERT/spaCy models pre-downloaded in the Docker image (not fetched at container start) to avoid slow first-run downloads during a live defence.

12. Development Roadmap

Aligned to the Agile sprint plan referenced in the project's Chapter Three (Table 3.2). Each sprint is roughly one to two weeks; adjust to the actual defence timeline.

Sprint	Focus	Deliverables
0	Project setup	Repo scaffolding (Section 3), Docker Compose, CI skeleton, database migrations for all 7 entities (Section 4), auth (login/JWT)
1	Document ingestion	Document upload API + UI, file storage, Stage 1-2 (ingestion, preprocessing) working end-to-end on sample files
2	Core NLP pipeline	Stages 3-6 (TF-IDF, NER, SBERT, BERTopic) implemented and unit-tested against fixtures; Celery task wiring
3	Scoring and overlap detection	Stage 7 scoring formula (Section 6), overlap detection against seeded core-curriculum corpus, Stage 8 output persistence
4	Recommendation dashboard	RecommendationDashboardView, ScoreBreakdown, OverlapBadge, accept/reject flow, session polling
5	Curriculum mapping and reports	CurriculumMapView + API, PDF/DOCX report generation
6	Admin and polish	AdminView (users, core curriculum repo, audit log), error handling, loading states, responsive/accessibility pass
7	Integration testing and UAT prep	Full integration test suite green, seed real UUY-CSC data, dry-run of the UAT task script (Appendix B) internally
8	UAT and evaluation	Run UAT with faculty sample, administer SUS, compute NLP benchmark metrics (Section 5 targets), feed results into project Chapter Four

Sprint	Focus	Deliverables
9	Deployment and defence prep	Deploy to staging/demo (Section 11), fix issues found in UAT, freeze for demo

Critical path note: Sprints 1–3 (the NLP pipeline itself) are the highest-risk, highest-effort work and should start immediately — the dashboard and mapping UI (Sprints 4–5) are comparatively low-risk CRUD work that can absorb schedule slippage better than the pipeline can. If time is short, cut Sprint 6 (Admin) scope first, not the pipeline or the UAT sprint, since UAT results are what Chapter Four’s Results and Discussion section reports on.